



**Course Title: DevOps Virtualization and Configuration
Management (INT332)
CI/CD Pipeline Project**

Submitted By:

Name: Bayarmaa

Registration Number: 12222441

Section: KO024

Roll No: 55

DECLARATION

I hereby declare that the project work entitled ("Title of the project") is an authentic record of my own work carried out as requirements of Project for the award of B.Tech Degree in _____CSE____(Programme Name) from Lovely Professional University, Phagwara, under the guidance of (Name of Faculty Mentor), during January to May 2025. All the information furnished in this project report is based on my own intensive work and is genuine.

Name of Student : Bayarmaa Bumandorj

Registration Number: 12222441

(Signature of Student /Bayarmaa/)

Date:17/4/2025

CERTIFICATE

This is to certify that the declaration statement made by the student is correct to the best of my knowledge and belief. He has completed this Project under my guidance and supervision. The present work is the result of his original investigation, effort and study. No part of the work has ever been submitted for any other degree at any University. The Project is fit for the submission and partial fulfillment of the conditions for the award of B.Tech Degree in _____ (Programme Name) from Lovely Professional University, Phagwara.

Signature and Name of the Mentor

Designation

School of Computer Science and Engineering,

Lovely Professional University,

Phagwara, Punjab.

Date :

1. INTRODUCTION

Indeed, this project has been titled DevOps-Based ToDo Application with CI/CD and Docker, which is one simple demo for the central principles of DevOps as an easy web-based ToDo list application. DevOps, one of the most recent changes in methodology of software development, testing, and deployment, is truly integrating the working and operation teams using automation tools and practices.

Using Jenkins, a CI/CD pipeline has been built to automate the entire development lifecycle further. As a result of the pipeline, the application is automatically built and tested for every code change, which makes changes to software into a faster and much more reliable delivery pipeline. Jenkins pulls the code from GitHub whenever changes are committed, installs dependencies, builds the application, runs tests, and then builds a Docker image. If the build is successful, Jenkins pushes the Docker image to DockerHub and runs the container on the target environment. This pipeline ensures reliable, repeatable, and fast software delivery with minimal manual effort.

The real motive of this project is to develop a complete automated software delivery pipeline with a frontend interface for users to manage their daily activities and a back-end server for task processing. The application is containerized using Docker to ensure consistent, replicable environments and portability between machines.

It should be made possible for the end-user to make use of automated software delivery pipeline through an interface with a front-end portal, while the backend will take care of the processing of various tasks.

The introduction outlines the aims and objectives of the project, preparatory for the specifics to follow. The passage enunciates the relevance of DevOps practices in modern software engineering and introduces technologies by means of which such practices are realized.

1.1. Project Architecture and Tech Stack

The entry-level paradigm where the mission and architectural breakdown of ToDo application get defined. Technicalities here underline picking Node.js for the backend and containerizing with Docker and CI/CD integration using GitHub Actions. These show how a combination of infrastructure works for deployment and continuous integration.

1.2. Benefits of Automation and Containerization

Another dimension of the project is its takeaway on automation and environment consistency. By using containers, the development and production environments remain as similar as possible such that environmental differences become a very low probable cause for bugs. This, in turn, eases deployment and consequently engenders better collaboration among team members who now go through the same containerized setup.

CI/CD, on the other hand, offers automated checks on the application after changes in any piece of code. This further strengthens the reliability of the whole system while also ensuring issues are picked up fast and feedback loops are established.

1.2.1. Role of CI/CD with Jenkins and GitHub

GitHub Actions provide the CI/CD tool that fits nicely with the version control system, allowing workflows for build, test, and deploy to be triggered with triggers automatically. Each change thus encounters a common pathway before merging or releasing, which ensures increased code quality.

Jenkins provides the CI/CD engine in this project. It integrates with GitHub for version control and triggers builds on code changes. Jenkins runs a pipeline that installs dependencies, runs tests, builds the app, and deploys it using Docker. This automation ensures reliability, consistency, and fast delivery.

1.2.2. Docker's Contribution to Deployment Simplicity

Docker is crucial when it comes to simplifying deployments. Instead of going through the mundane steps of server configuration, Docker simply packages the whole application with all its dependencies into a single image. This image then runs consistently across development, staging, and production environments. Further scaling and orchestration are supported when combined with additional tools like Kubernetes.

2. SCOPE OF THE STUDY (PROBLEM STATEMENT)

This project aims to solve problems associated with the classical method of deploying software: manual configuration, inconsistent environments, and delays in delivery cycles. The project

seeks to develop a continuous and automated deployment pipeline, ensuring consistency and efficiency throughout the complete software development life cycle, applied with the following DevOps practices: Containerization and CI/CD pipelines.

The ToDo application is developed as a practical case study for demonstration of these DevOps practices. The scope includes design and development of front-end and back-end, containerizing application with Docker, and automating the build and deploy process using GitHub Actions.

3. EXISTING SYSTEM

3.1 Introduction

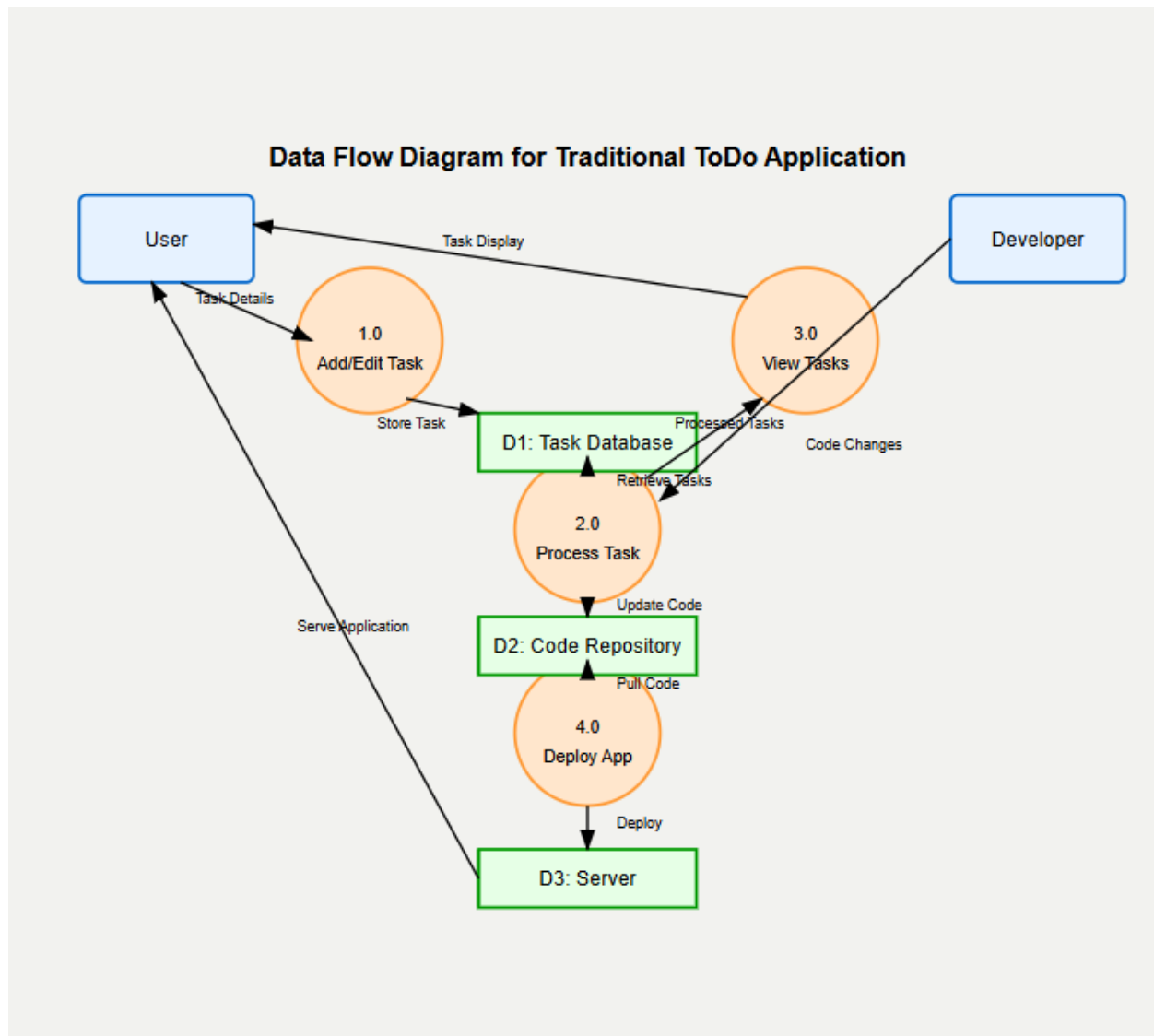
Many existing software development setups are manual setups where code is built and deployed. These setups make the development work slow and prone to errors. Developers also find it difficult to reproduce the same environment in different machines.

3.2 Existing Software

Traditional ToDo applications either get hosted manually or with the most basic automated deployment tools. These systems do not have automated pipelines for consistency between environments, revealing constant unpredictable behavior among development, staging, and production.

3.3 DFD for Present System

The **Data Flow Diagram (DFD)** below represents the current manual or semi-automated process followed in traditional ToDo applications before implementing CI/CD and containerization.



3.4 What's New in the System to Be Developed

This project will introduce automation through Docker and CI/CD, with the entire application stack containerized. Changes in the codebase will trigger automated builds and deployment by GitHub Actions, ensuring fast and reliable software delivery.

4. PROBLEM ANALYSIS

4.1 Product Definition

An uncomplicated and functioning ToDo application that is fully automated in terms of the development, testing, and deployment enters the world using modern DevOps tools and techniques.

4.2 Feasibility Analysis

- **Technical feasibility:** It uses popular open-source tools like Node.js, Docker, and GitHub Actions.
- **Operational feasibility:** It is easy to work with via Docker Compose and web interface.
- **Economic feasibility:** Infrastructure cost is zero at best due to its running on local setups and at free by GitHub services.

4.3 Project Plan

1. Frontend & backend application setup
2. Dockerization of application
3. CI/CD setup via Jenkins
4. Triggering and testing of the deployment pipeline
5. Documentation of process & its output

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

This section describes the software and system requirements for the successful execution of the project.

5.2 General Description

The application consists of two components:

- A frontend constructed with HTML, CSS, and JavaScript
- A backend constructed with Node.js and Express

5.3 Specific Requirements

- Node.js (v18 or higher)
- Docker (Engine and Compose)
- Git and GitHub account
- VS Code or any code editor
- Web browser (Chrome/Firefox)
- Jenkins

6. DESIGN

6.1 System Design

In this application segment, clients use the service offered by Nginx as the main interface with the back-end being an actual Node.js server.

The frontal part of the entire application communicates with the backend, which basically manages the requests coming from the client server towards that Node.js server.

The auditorium can now receive, take care of, and return back the information related to the task.

6.2 Design Notations

- Arrows indicate the directions of flow for the data
- Rectangles denote modules (Frontend, Backend)
- A cylinder denotes data (Task list array).

6.3 Detailed Design

These are the two endpoints exposed by the back end:

- GET /todos — to receive the list of planned activities
- POST /todos — to add a new task

Frontend makes calls by fetch into these two APIs and renders the response into the browser.

6.4 Flowcharts



6.5 Pseudo Code

On user input:

Capture text

Send POST request to backend

Update task list on success

7. TESTING

7.1 Functional Testing

Functional Testing as the counts increase- it deals with adding tasks via frontends, viewing the total number of newly added tasks.

7.2 Structural Testing

These are API endpoints which correspond with expected returns.

7.3 Levels of Testing

- Manual on browser
- Build success CI check

7.4 Testing the Project

Testing was done using local Docker Compose setup and browser interaction.

8. IMPLEMENTATION

8.1 Implementation of the Project

The project was implemented on the following lines:

- Node.js for backend logic
- HTML/CSS/JS for the UI
- Docker for containerizing the application
- Jenkins for CI/CD

8.2 Conversion Plan

There is no need for conversions since the application is new.

8.3 Post-Implementation and Software Maintenance

The fact that it's lighter and easy to maintain through the consistency ensured by images means that future updates could easily be integrated using CI/CD.

9. PROJECT LEGACY

9.1 Current Status of the Project

The ToDo application is up and running and also dockerised and has been configured for CI/CD using GitHub Actions.

9.2 Remaining Areas of Concern

- Persistent storage of a database (e.g., MongoDB)
- Authentication mechanism for users.

9.3 Technical and Managerial Lessons Learnt

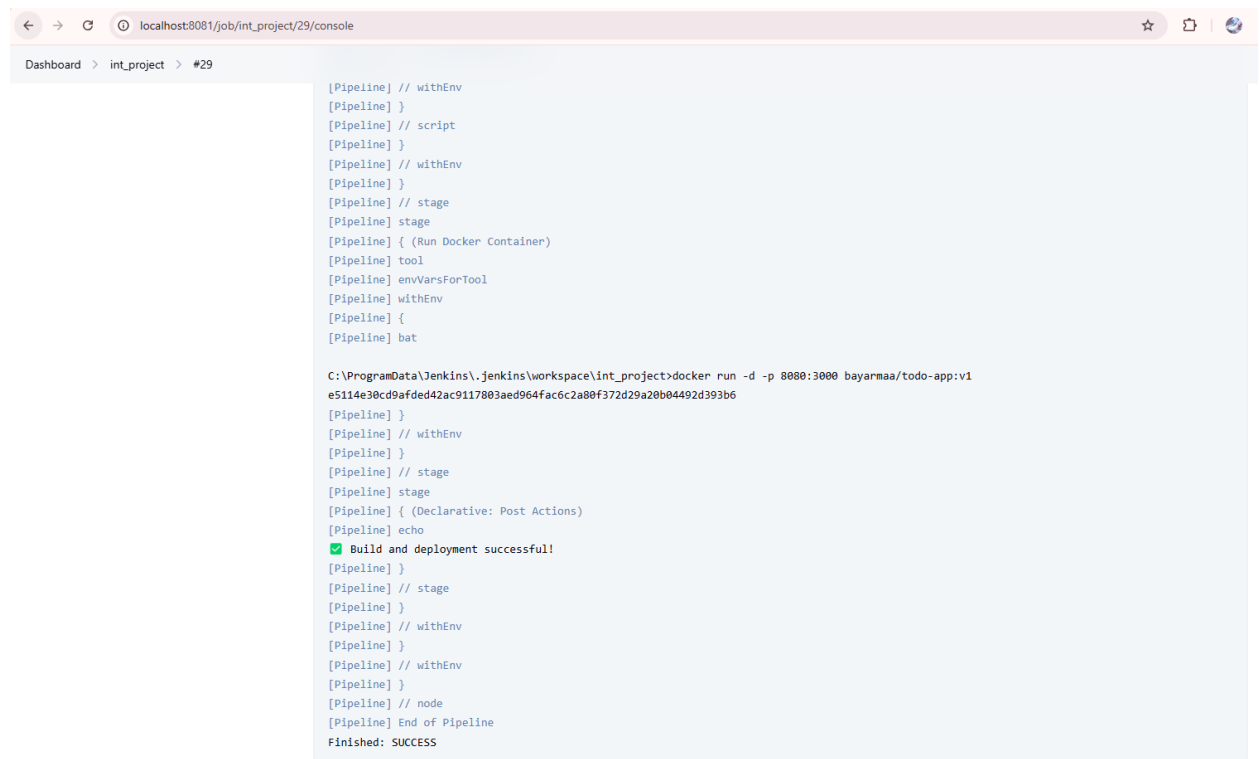
- CI/CD pipelines have increased efficiency for developers.
- Deployment is being simplified with Docker.
- Building automation through GitHub Actions is powerful.

10. SOURCE CODE / SYSTEM SNAPSHOTS

(Add screenshots for the following with appropriate captions:)

- CI/CD setup via Jenkins

Link: http://localhost:8081/job/int_project/29/console

A screenshot of a web browser displaying the Jenkins console output for a pipeline named 'int_project' at build #29. The browser's address bar shows 'localhost:8081/job/int_project/29/console'. The console output is a log of pipeline steps, including 'withEnv', 'script', 'stage', and 'docker run'. A green checkmark icon and the text 'Build and deployment successful!' are visible in the log. The log ends with 'Finished: SUCCESS'.

```
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Docker Container)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] bat

C:\ProgramData\Jenkins\workspace\int_project>docker run -d -p 8080:3000 bayarmaa/todo-app:v1
e5114e30cd9afded42ac9117803aed964fac6c2a80f372d29a20b04492d393b6
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
✅ Build and deployment successful!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

- Docker Compose up in terminal

```
}
adduser@DESKTOP-HS81N044:/mnt/c/Users/surface/OneDrive/Documents/3rd_year/INT_332/ca2/todo-app/backend$ docker logs e5114e30cd9a
Server running on port 3000
adduser@DESKTOP-HS81N044:/mnt/c/Users/surface/OneDrive/Documents/3rd_year/INT_332/ca2/todo-app/backend$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
e5114e30cd9a   bayarmaa/todo-app:v1               "docker-entrypoint.s..." 18 minutes ago Up 17 minutes  0.0.0.0:8080->3000/tcp             compassionate_mendeleev
adduser@DESKTOP-HS81N044:/mnt/c/Users/surface/OneDrive/Documents/3rd_year/INT_332/ca2/todo-app/backend$ docker logs e5114e30cd9a
Server running on port 3000
adduser@DESKTOP-HS81N044:/mnt/c/Users/surface/OneDrive/Documents/3rd_year/INT_332/ca2/todo-app/backend$
```

Q Search for images, containers, volumes, extensions... Ctrl+K



compassionate_mendeleev

<  bayarmaa/todo-app:v1

e5114e30cd9a

8080:3000

STATUS

Running (22 minutes ago)

Logs

Inspect

Bind mounts

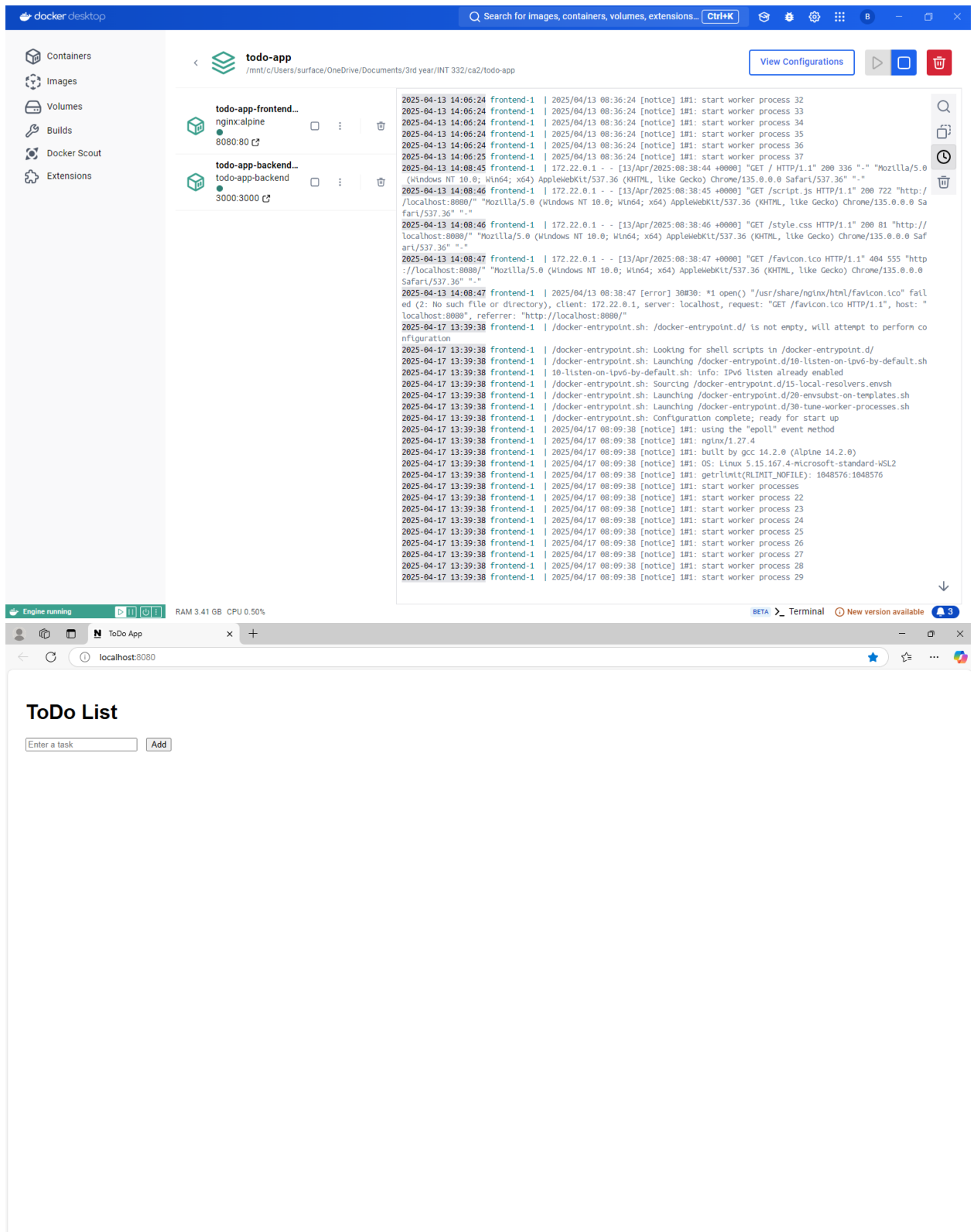
Exec

Files

Stats

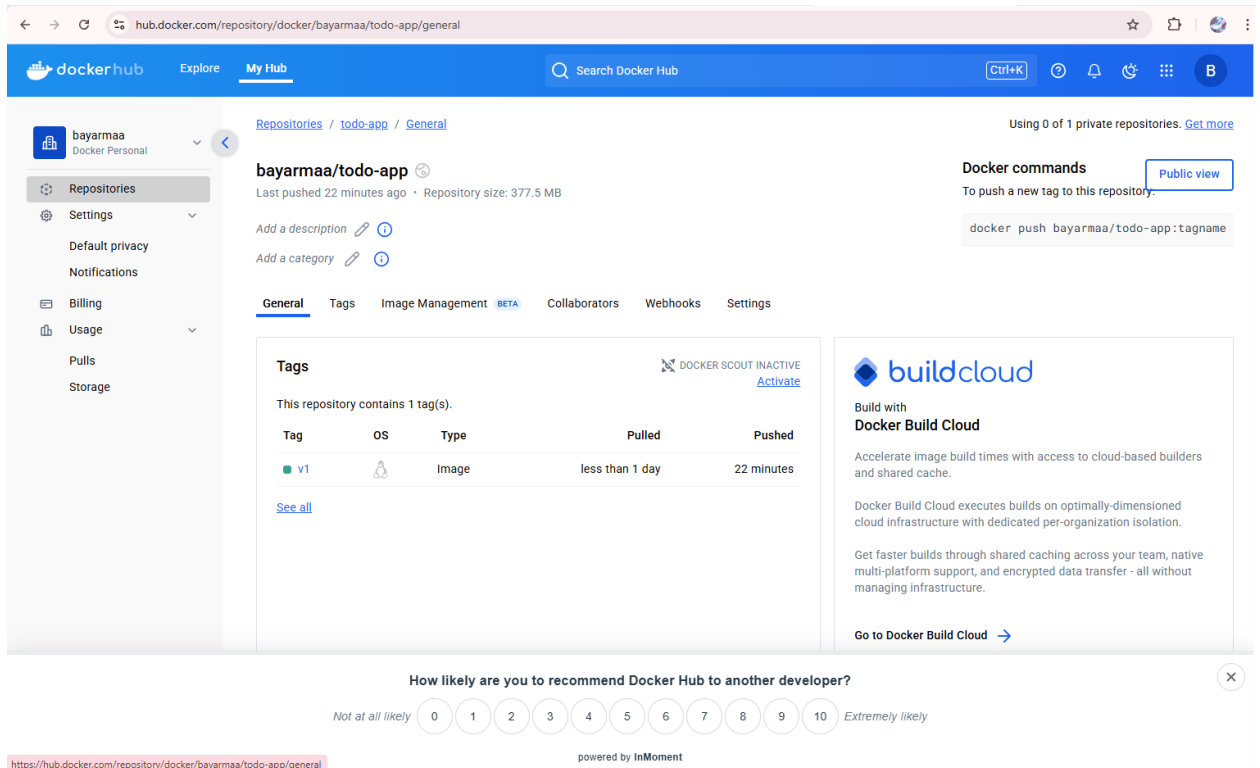
2025-04-16 07:47:45 Server running on port 3000



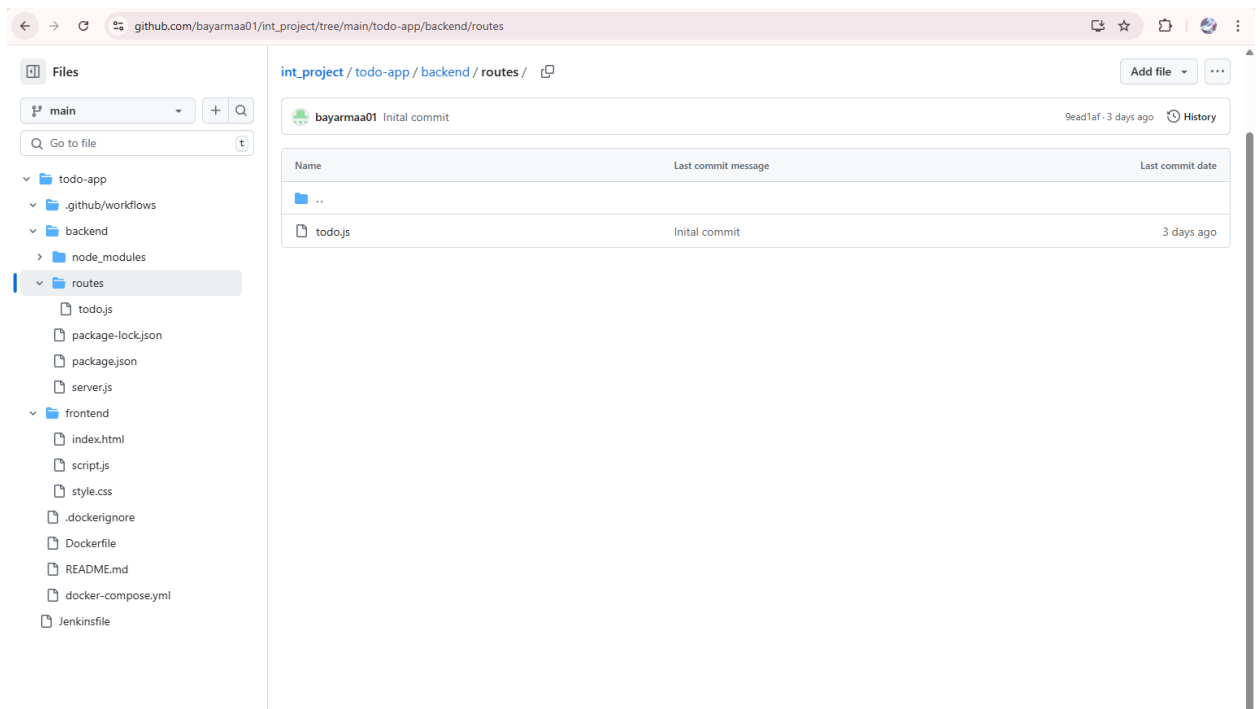
- Docker Hub

Link: <https://hub.docker.com/bayarmaa/todo-app>



- Frontend UI in browser
- GitHub

Link: https://github.com/bayarmaa01/int_project.git



```
1 pipeline {
2   agent any
3
4   tools {
5     nodejs 'Node20'
6   }
7
8   stages {
9     stage('Clone Repo') {
10      steps {
11        git branch: 'main', url: 'https://github.com/bayarmaa01/int_project.git'
12      }
13    }
14
15    stage('Install Backend Dependencies') {
16      steps {
17        dir('todo-app/backend') {
18          bat 'npm install'
19        }
20      }
21    }
22
23    stage('Build Docker Image') {
24      steps {
25        bat 'docker build -t bayarmaa/todo-app:v1 ./todo-app'
26      }
27    }
28
29    stage('Push to DockerHub') {
30      steps {
31        script {
32          docker.withRegistry('', 'dockerhub-creds') {
33            bat 'docker push bayarmaa/todo-app:v1'
34          }
35        }
36      }
37    }
38
39    stage('Run Docker Container') {
40      steps {
41        bat 'docker run -d -p 8080:3000 bayarmaa/todo-app:v1'
42      }
43    }
44  }
45
46  post {
47    failure {
48      echo '❌ Build Failed. Check logs for details.'
49    }
50    success {
51      echo '✅ Build and deployment successful!'
52    }
53  }
54 }
```

11. BIBLIOGRAPHY

- <https://nodejs.org/en/docs/>
- <https://docs.docker.com/>
- <https://docs.github.com/en/actions>
- <https://www.jenkins.io/>